

Azure Deploy (Container Apps + Azure SQL)

This document records the exact Azure resources and command flow used to deploy Finly.

Resources used

- Subscription: Azure subscription 1
- Resource group: finly-rg
- Location: centralindia
- Container Apps environment: finly-env
- Log Analytics workspace: finly-logs
- SQL server: finlysql29987
- SQL database: finlydb
- Container App: finly-app
- Container image: docker.io/nikhil251200/finly:latest
- Public URL: <https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io>

1) Prerequisites and provider registration

```
az account show
az provider register --namespace Microsoft.Sql
az provider show --namespace Microsoft.Sql --query registrationState -o tsv
```

2) Build and push Docker image

Important: Container Apps required linux/amd64 image.

```
docker buildx build --platform linux/amd64 -t nikhil251200/finly:latest --push .
```

3) SQL server password update, database creation, and firewall

```
RG="finly-rg"
SQL_SERVER="finlysql29987"
DB_NAME="finlydb"
```

```
# generate a strong SQL password locally
```

```
SQL_PASSWORD="$(python3 -c 'import secrets,string; chars=string.ascii_letters+string.digits+string.punctuation; print('".join(secrets.choice(chars) for _ in range(32))')")"
```

```
az sql server update -g "$RG" -n "$SQL_SERVER" -p "$SQL_PASSWORD"
az sql db create -g "$RG" -s "$SQL_SERVER" -n "$DB_NAME" --edition Basic
```

```
# allow Azure services + current local IP
```

```
az sql server firewall-rule create -g "$RG" -s "$SQL_SERVER" -n AllowAzure --start-ip-address 0.0.0.0 --end-ip-address 255.255.255.255
MYIP="$(curl -s https://ifconfig.me)"
az sql server firewall-rule create -g "$RG" -s "$SQL_SERVER" -n AllowMyCurrentIp --start-ip-address $MYIP --end-ip-address $MYIP
```

4) Create Container App

```
RG="finly-rg"
LOCATION="centralindia"
APP_NAME="finly-app"
ENV_NAME="finly-env"
IMAGE="docker.io/nikhil251200/finly:latest"
SQL_SERVER="finlysql29987"
SQL_ADMIN="finlyadmin"
DB_NAME="finlydb"

JWT_SECRET="$(python3 -c 'import secrets; print(secrets.token_urlsafe(48))')"

az containerapp create \
  -g "$RG" \
  -n "$APP_NAME" \
  --environment "$ENV_NAME" \
  --image "$IMAGE" \
  --ingress external \
  --target-port 8000 \
  --min-replicas 0 \
  --max-replicas 1 \
  --cpu 0.5 \
  --memory 1.0Gi \
  --secrets db-password="$SQL_PASSWORD" jwt-secret="$JWT_SECRET" \
  --env-vars \
    APP_ENV=production \
    DB_SERVER="$SQL_SERVER.database.windows.net" \
    DB_PORT=1433 \
    DB_NAME="$DB_NAME" \
    DB_USER="$SQL_ADMIN" \
    DB_PASSWORD=secretref:db-password \
    DB_ENCRYPT=yes \
    DB_TRUST_SERVER_CERTIFICATE=no \
    JWT_SECRET=secretref:jwt-secret \
    CORS_ALLOWED_ORIGINS="https://finly-app.lemonflower-e7039555.centralindia.azurecontainer
```

5) Initialize schema

az containerapp exec was not usable in this CLI environment, so schema init was run using the container image directly:

```
docker run --rm --platform linux/amd64 \
  -e APP_ENV=production \
  -e DB_SERVER="finlysql29987.database.windows.net" \
  -e DB_PORT=1433 \
  -e DB_NAME="finlydb" \
```

```

-e DB_USER="finlyadmin" \
-e DB_PASSWORD="$SQL_PASSWORD" \
-e DB_ENCRYPT=yes \
-e DB_TRUST_SERVER_CERTIFICATE=no \
-e JWT_SECRET="temp-init-secret" \
-e CORS_ALLOWED_ORIGINS="*" \
nikhil251200/finly:latest \
python -c "from app.core.db_init import init_database_schema; init_database_schema(); print"

```

6) Rotate/update app secret after SQL password changes

```

az containerapp secret set -g "$RG" -n "$APP_NAME" --secrets db-password="$SQL_PASSWORD"
az containerapp update -g "$RG" -n "$APP_NAME" --set-env-vars DEPLOYED_AT="$(date +%s)" DB_P

```

7) Verification commands

```

az containerapp show -g finly-rg -n finly-app --query "{fqdn:properties.configuration.ingressEndpoint}"
curl -sS https://finly-app.lemonflower-e7039555.centralindia.azurecontainerapps.io/health
curl -sS -o /dev/null -w "%{http_code}" https://finly-app.lemonflower-e7039555.centralindia

```

Notes

- If deployment fails with image platform error, rebuild with `--platform linux/amd64`.
- If root/login pages return stale HTML, hard refresh browser cache.
- If SQL auth fails after password rotation, resync `db-password` secret and trigger a new revision with `DEPLOYED_AT` env update.